Le perceptron

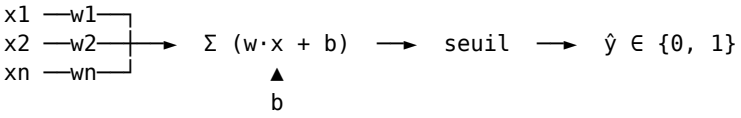
Fiche de synthèse : composantes, entraînement, limites et cas d'usage

1. Définition

Le perceptron, proposé par Frank Rosenblatt en 1958, est le modèle de neurone artificiel le plus simple. C'est un **classifieur binaire linéaire** : il sépare les données en deux classes par une droite (2D), un plan (3D) ou un hyperplan (n dimensions).

2. Composantes

Composante	Rôle
Entrées $x_1, \ldots, x_n$	Caractéristiques (features) de l'exemple
Poids $w_1, \ldots, w_n$	Importance de chaque entrée, apprise pendant l'entraînement
Biais b	Décale la frontière de décision (elle ne passe pas forcément par l'origine)
Somme pondérée	$z = w_1x_1 + \ldots + w_nx_n + b = w \cdot x + b$
Fonction d'activation	Seuil (Heaviside) : $\hat{y} = 1$ si $z \geq 0$, sinon 0
Sortie $\hat{y}$	Classe prédite



3. Entraînement

Apprentissage **supervisé** : chaque exemple x est fourni avec son étiquette réelle y.

- Initialiser les poids et le biais (à 0 ou avec de petites valeurs aléatoires).
- Pour chaque exemple (x, y), calculer $\hat{y} = \text{seuil}(w \cdot x + b)$.
- Si $\hat{y} \neq y$, corriger les paramètres avec la règle ci-dessous ; sinon, ne rien changer.
- Répéter sur plusieurs **époques** jusqu'à ce qu'il n'y ait plus d'erreur (ou un nombre maximal d'itérations).

$$\mathbf{w_i \leftarrow w_i + \eta \cdot (y - \hat{y}) \cdot x_i \quad b \leftarrow b + \eta \cdot (y - \hat{y})}$$

η est le taux d'apprentissage. **Intuition :** si le modèle prédit 0 au lieu de 1, on augmente les poids des entrées actives pour pousser z vers le positif, et inversement. **Théorème de convergence :** si les données sont linéairement séparables, l'algorithme trouve une frontière correcte en un nombre fini d'étapes.

4. Limite : le problème du XOR

Le perceptron ne peut pas apprendre une fonction **non linéairement séparable**. Avec le XOR, aucune droite ne sépare (0,1) et (1,0) de (0,0) et (1,1). Minsky et Papert l'ont démontré en 1969. **Solution :** empiler plusieurs couches avec des activations non linéaires et dérivables (sigmoïde, ReLU…) pour obtenir le **perceptron multicouche (MLP)**, entraîné par rétropropagation du gradient. C'est la base du deep learning.

5. Cas d'usage

Perceptron simple	Perceptron multicouche (MLP)
Portes logiques (AND, OR, NOT)	Reconnaissance de chiffres manuscrits (MNIST)
Classification binaire de données linéairement séparables (spam basique, accepté/refusé)	Détection de fraude bancaire
Apprentissage en ligne (mise à jour exemple par exemple)	Prédiction de churn, scoring de crédit
Baseline rapide avant des modèles plus complexes	Régression (prédiction de prix)
Outil pédagogique pour comprendre les réseaux de neurones	Couches de classification finales des CNN et Transformers

6. Exemple : prédire la réussite d'un module

On veut prédire si un étudiant **réussit (1)** ou **échoue (0)** à un module à partir de deux entrées : sa **note théorique** x_1 et sa **note pratique** x_2 (sur 20). Le jeu de données contient 80 étudiants fictifs (49 réussites, 31 échecs). Les notes sont divisées par 20 pour être ramenées entre 0 et 1.

Code Python

```
import numpy as np

class Perceptron:
    def __init__(self, lr=0.1, epochs=100):
        self.lr, self.epochs = lr, epochs

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1]); self.b = 0.0
        for epoch in range(self.epochs):
            erreurs = 0
            for xi, yi in zip(X, y):
                erreur = yi - self.predict(xi)       # -1, 0 ou +1
                if erreur != 0:
                    self.w += self.lr * erreur * xi
                    self.b += self.lr * erreur
                    erreurs += 1
            if erreurs == 0:                         # convergence
                break

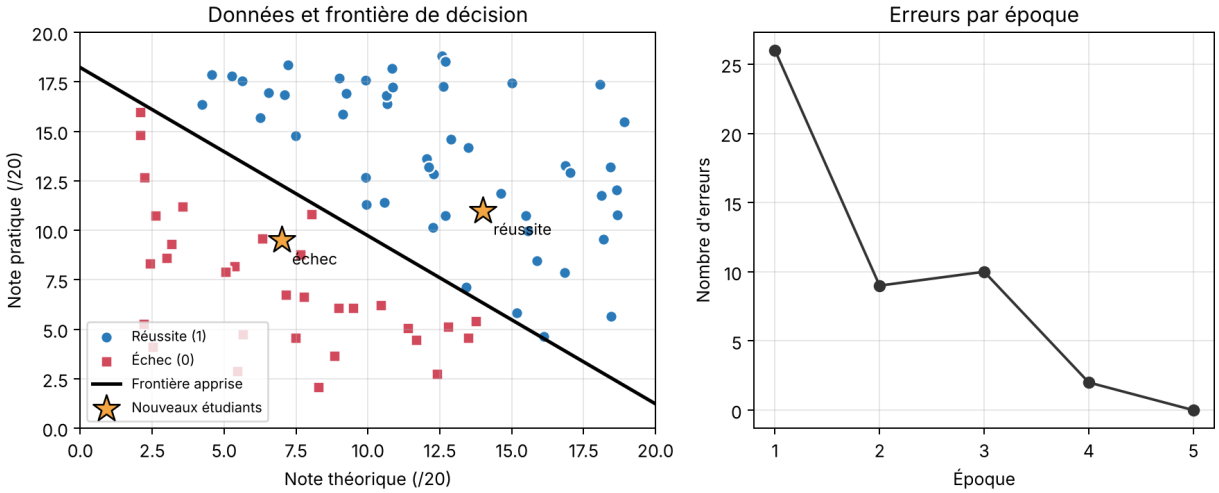
    def predict(self, X):
        return np.where(X @ self.w + self.b >= 0, 1, 0)

X_norm = X / 20                      # notes ramenées dans [0, 1]
model = Perceptron(lr=0.1)
model.fit(X_norm, y)
model.predict(np.array([[14, 11], [7, 9.5]]) / 20)
```

Résultats

Époque	Erreurs	w_1	w_2	b
1	26	0.227	0.175	-0.20
2	9	0.224	0.228	-0.30
3	10	0.317	0.270	-0.30
4	2	0.280	0.329	-0.30
5	0	0.280	0.329	-0.30

Le perceptron converge en **5 époques** et classe correctement **100%** des exemples d'entraînement. Les deux poids sont positifs et proches : le modèle a découvert que les deux notes comptent presque autant. La frontière apprise a pour équation $\mathbf{0.280 \cdot x_1 + 0.329 \cdot x_2 - 0.30 = 0}$ (notes normalisées).



À gauche : les étudiants et la droite apprise par le perceptron. À droite : le nombre de corrections diminue au fil des époques jusqu'à atteindre 0.

Prédictions sur de nouveaux étudiants

Étudiant	Théorie	Pratique	z = w·x + b	Prédiction
A	14	11	+0.077	Réussite (1)
B	7	9,5	-0.046	Échec (0)

Le signe de z décide de la classe. Cet exemple fonctionne parce que les données sont **linéairement séparables**. Si la réussite dépendait d'une règle non linéaire (par exemple « une seule des deux notes au-dessus de 10 », équivalent du XOR), le perceptron simple ne convergerait jamais.

En résumé

Le perceptron fait une somme pondérée de ses entrées, ajoute un biais, puis applique un seuil. Il apprend en corrigeant ses poids à chaque erreur. Il est simple et rapide, mais limité aux problèmes linéairement séparables. Ses descendants multicouches sont au cœur de l'IA moderne.